Interview Prep Guidelines 5

Interview Prep Guidelines 5

Part 5 (Questions 81–100) – Performance, Optimization & Senior Front-End Architecture

These questions are very common for **Senior Front-End** interviews because they test experience rather than syntax.

---

81. How do you optimize a React application?

Answer

I first identify the bottleneck using React DevTools Profiler, Lighthouse, or browser performance tools. Then I optimize only where necessary.

Common optimizations:

- Use `React.memo()` for expensive components.
- Use `useMemo()` for expensive calculations.
- Use `useCallback()` for stable function references.
- Code splitting with `React.lazy()` or dynamic imports.
- Virtualize large lists.
- Lazy-load images.
- Reduce unnecessary state updates.
- Move state closer to where it’s used.
- Avoid unnecessary Context updates.
- Optimize bundle size.

Senior Tip

> *Don’t optimize everything. Measure first, optimize second.*

---

82. What causes unnecessary re-renders?

Common causes:

- Parent re-renders.
- New object literals.

```
        <User style={{color:"red"}} />
```

Every render creates a new object.

Instead:

```
        const style = {color: "red"};
```

New functions:

```
        <Button onClick={() => save()} />
```

Better:

```
        const handleClick = useCallback(save, []);
```

Other causes:

- Context updates.
- Large global state.
- Incorrect dependency arrays.

---

83. When should you use React.memo()?

`React.memo()` skips rendering when props haven't changed.

```
        const User = React.memo(function User() {
            ...
        });
```

Use it when:

- Component renders frequently.
- Rendering is expensive.
- Props rarely change.

Don't use it everywhere because prop comparison also has a cost.

---

84. When should you use useMemo()?

Memoize expensive calculations.

```
        const sortedUsers = useMemo(() => {
            return users.sort(...)
        }, [users]);
```

Good:

- Sorting
- Filtering
- Large calculations

Bad:

```
        const fullName = useMemo(() => first + last, [first, last]);
```

String concatenation is cheap, so `useMemo` adds unnecessary overhead.

---

85. When should you use useCallback()?

Memoize functions passed to child components.

```
const handleDelete = useCallback(() => {
    ...
}, []);
```

Useful with:

```
React.memo()
```

Avoid using it for every function, as it also has overhead.

---

86. What is Code Splitting?

Instead of loading the whole application:

```
1 MB JavaScript
```

Load only what's needed.

React:

```
const Admin = React.lazy(() => import("./Admin"));
```

Next.js:

```
const Chart = dynamic(() => import("./Chart"));
```

Benefits:

- Faster initial load.
- Smaller bundles.
- Better Lighthouse score.

---

87. What is Lazy Loading?

Load resources only when needed.

Examples:

Images

```
<img loading="lazy">
```

Components

```
React.lazy()
```

Routes

```
        dynamic()
```

Benefits:

- Faster startup.
- Lower bandwidth usage.

---

88. What is Virtualization?

Rendering 100,000 rows is slow.

Instead: render only what’s visible.

Libraries:

- react-window
- react-virtualized

Instead of:

```
100000 DOM Nodes
```

Render:

```
40 DOM Nodes
```

Huge performance improvement.

---

89. What are Web Vitals?

Google’s performance metrics.

LCP (Largest Contentful Paint)

How quickly the main content appears.

Good:

```
< 2.5 seconds
```

INP (Interaction to Next Paint)

Measures how responsive the page feels after user interactions.

Good:

```
< 200 ms
```

CLS (Cumulative Layout Shift)

Measures visual stability.

Good:

```
< 0.1
```

Improve CLS by reserving space for images and ads.

---

90. How do you reduce bundle size?

Methods:

- Tree shaking.
- Code splitting.
- Dynamic imports.
- Remove unused libraries.
- Import only what’s needed.

Bad:

```
        import _ from "lodash";
```

Better:

```
        import debounce from "lodash/debounce";
```

Or use native JavaScript where possible.

---

91. How would you debug a slow React application?

Steps:

1. React DevTools Profiler.
2. Chrome Performance tab.
3. Lighthouse.
4. Check unnecessary renders.
5. Analyze bundle size.
6. Look for expensive loops.
7. Optimize API requests.
8. Virtualize lists.
9. Memoize only if profiling shows it helps.

---

92. How do you optimize API calls?

Methods:

- Debounce search.
- Cache responses.
- Parallel requests.

```
        await Promise.all([
            fetchUsers(),
            fetchPosts()
        ]);
```

- Pagination.
- Infinite scrolling.
- Avoid duplicate requests.
- Retry failed requests where appropriate.

---

93. Explain Browser Caching.

Browser stores:

- Images
- CSS
- JS
- Fonts

Headers:

```
Cache-Control
ETag
Expires
```

Benefits:

- Faster loading.
- Less bandwidth.
- Lower server load.

---

94. What is CDN?

Content Delivery Network.

Instead of:

```
Bangladesh  →  USA Server
```

Use:

```
Bangladesh  →  Singapore CDN  →  USA Origin
```

Benefits:

- Lower latency.
- Faster downloads.
- DDoS protection.
- Global scalability.

---

95. Explain State Management options.

Local State

```
useState()
```

For component-specific state.

Context API

Good for:

- Theme
- Authentication
- Language

Not ideal for frequently changing global state because all consumers may re-render.

Redux Toolkit

Best for:

- Large applications
- Predictable updates
- DevTools
- Middleware

Zustand

Simple. Small API. Minimal boilerplate.

Great for medium-sized applications.

---

96. Context vs Redux

Context	Redux
Built into React	External library
Simple	Scalable
Limited tooling	Excellent DevTools
Can trigger broad re-renders	Fine-grained updates with selectors

Interview answer:

> *Context is ideal for low-frequency global state like themes or authentication. Redux or Zustand is often a better fit for complex, frequently updated application state.*

---

97. How do you handle errors in React?

Use Error Boundaries.

```
class ErrorBoundary extends React.Component {
    ...
}
```

They catch:

- Rendering errors
- Lifecycle errors
- Constructor errors

They **do not** catch:

- Event handler errors
- Async errors (`fetch`, `setTimeout`)
- Server-side rendering errors

For async code, use `try/catch` or error handling in your data-fetching layer.

---

98. How do you secure a frontend application?

Best practices:

- Store tokens in **HttpOnly cookies** when possible.
- Sanitize user input.
- Prevent XSS.
- Enable CSP (Content Security Policy).
- Use HTTPS.
- Avoid exposing secrets.
- Validate input on both client and server.
- Protect against CSRF when using cookies.
- Escape HTML before rendering user-generated content.

---

99. Describe your ideal React project structure.

```
src/
  components/
  features/
  hooks/
  services/
  pages/
  utils/
  types/
  constants/
  assets/
  layouts/
```

For larger apps, organize by **feature** rather than by file type.

Example:

```
features/
  auth/
  dashboard/
  users/
  products/
```

Each feature contains its own components, hooks, services, and tests, improving scalability and maintainability.

---

100. How would you design a scalable frontend architecture?

Answer

For a large enterprise application, I would:

- Organize code by feature.
- Use reusable UI components.
- Separate business logic from presentation.
- Use TypeScript throughout.
- Use Redux Toolkit or Zustand only for truly global state.
- Use React Query/TanStack Query (or Next.js data fetching where appropriate) for server state.
- Keep local UI state local.
- Lazy-load routes and heavy components.
- Optimize rendering with profiling rather than premature optimization.
- Add automated testing (unit, integration, E2E).
- Enforce linting, formatting, and code review standards.
- Use CI/CD with staging and production environments.
- Monitor performance with Lighthouse and Core Web Vitals.

A concise senior answer:

> *"I focus on separation of concerns, feature-based organization, reusable components, predictable state management, performance optimization, automated testing, and maintainable code. The goal is to build an application that's easy to scale as both the team and the product grow."*

☆ Bonus: Questions They May Ask Based on Your Experience

Given your background, be ready for these:

How did you optimize your last React application?

Talk about:

- Code splitting
- Lazy loading
- Memoization where profiling showed benefits
- API caching
- Reducing unnecessary re-renders

Tell me about a difficult production bug.

Use the STAR format:

- **Situation**
- **Task**
- **Action**
- **Result**

How do you review code?

A strong answer:

- Verify correctness.
- Look for edge cases.
- Check readability and maintainability.
- Ensure consistent naming.
- Evaluate performance implications.
- Confirm proper error handling.
- Provide constructive feedback rather than just pointing out issues.

How do you mentor junior developers?

Discuss:

- Pair programming.
- Code reviews.
- Architecture discussions.
- Explaining the “why,” not just the “how.”
- Encouraging questions and ownership.
- Sharing best practices and learning resources.

These 100 questions cover the majority of topics commonly discussed in senior React/Next.js interviews. For a **Senior Front-End Developer** role, interviewers typically value your reasoning, trade-off analysis, and real-world experience even more than memorized definitions.